

Supplementary material for “dinara-align: exact pairwise alignment specialised to the target processor”

Konstantinos Kyriakidis and Benedict Paten

This supplement provides implementation details, per-change measurements and additional results that the main text summarises. Section S1 describes techniques not covered in the main text, Section S2 lists the measured effect of each change, Section S3 gives the complete results for gap-affine costs and the other alignment modes, and Sections S4 and S5 describe robustness measures and approaches that were evaluated and not adopted. Unless stated otherwise, the measurements in Sections S1, S2, S4 and S5 were taken when each change was introduced, on the machine indicated, and compare the change with the preceding version; the results in the main text and in Section S3 were obtained at commit `ebe290f`. Two machines were used: an Intel Core i9-7900X (Skylake-X, AVX-512) and an Apple M2 (NEON).

S1 Implementation details

S1.1 Unit costs

Engine selection. The diagonal transition receives a step budget proportional to the cost of the band it would replace; on the Skylake-X a step costs about 1.3 ns and a band column about 6 ns. Inexact seeds replace exact seeds when fewer than 20, 30 or 40% of exact seeds chain on AVX-512, AVX2 and NEON, respectively: the wider the vector unit, the cheaper the bit-parallel band relative to the scalar construction of inexact seeds. In the gap-affine engine, a pair whose wavefront is expected to cover more than a quarter of its matrix is assigned to the anti-diagonal sweep, a wavefront step costing about 1.5 ns and a sweep cell about 0.4 ns. All thresholds are fixed at compile time for each instruction set, so that no dispatch occurs at run time.

Register and memory layout of the sweep. An empty inline-assembly barrier prevents the compiler from reassociating the regrouped recurrence back to its original form. Vertical differences remain in registers across a word’s columns, and the horizontal differences of successive columns are stored at distinct addresses, so that no load waits on a preceding store. Bands of five to seven words combine a four-lane vector with scalar words.

Symbols other than ACGT. N and other non-ACGT symbols are encoded in a third bit plane. Each tile uses only the planes its symbols require: two when neither its columns nor any row contains such a symbol, the third plane of the rows as a mask when only rows contain one, and all three otherwise. Sequences are encoded 16 bytes or eight rows at a time, and only chunks that contain such a symbol are processed byte by byte. This made reads containing N 7 to 9% faster on both machines. A seed that contains N is not counted, so that pairs containing N can still use seeds; this made the ultra-long reads 2.5 to 3 times faster on the M2 and 1.4 to 1.7 times faster on the Skylake-X.

Tile width. Tiles are 256 columns wide, as in A*PA2, or 64 columns wide in bands of fewer than 1,024 rows.

Accepting an extrapolation. The diagonal-transition estimate is abandoned once it has spent one twenty-fourth of the cost of the cheapest band. An extrapolation sets the band only when the single-wavefront extrapolation from the first edits and the extrapolation of the longer search agree to within one third; otherwise the first bound is set 256 above the known lower bound, and each retry at most doubles it. A later checkpoint never lowers the bound below the previous bound plus one quarter, and a failed round continues from the bound at which it ended rather than the one at which it started. Before these rules, a 30 kbp SARS-CoV-2 pair at distance 49 had been extrapolated to 14,448 and computed over the full matrix.

When they were introduced, these rules reduced the mean time on the ultra-long reads with genetic variation from 4.33 s, with 13 of 15 reads completed within the time limit, to 754 ms. At the evaluated commit the seed heuristic sets the first bound on long reads, and switching the agreement test off changes the ultra-long reads by less than 1% but slows the 10 kbp reads by 17% (main text, Effect of each technique).

Retries with seeds. When seeds are used, a retry sets the bound one quarter of the extrapolated increase beyond the estimate; with one half, a round had failed after 90% of the columns of a 527 kbp read. A seeded round that fails within the first eighth of its columns does not extrapolate; instead, its margin over the heuristic bound at the origin increases fourfold, as in A*PA2. Previously, an 884 kbp read had failed at column 1,280 and been extrapolated to 125,000 for a distance of 75,000. Only the first round revises its bound at the checkpoints; when later rounds also lowered their bounds, they failed late, four times on one 897 kbp read.

Traceback. A tile on which the forward search exceeds its limit is recomputed over a window of rows above the traced cell, starting at four words and doubling. The top boundary of the window is initialised to +1, so that its scores correspond to real paths. A 600 kbp nanopore pair at 13% divergence was aligned in 617 ms instead of 1.38 s, of which 840 ms had been spent recomputing tiles of 900 words. Recomputation buffers are reused without initialisation, and tile boundaries are allocated once and written through pointers; loading the boundaries had accounted for a quarter of the traceback time. The CIGAR string is produced by reversing moves 16 at a time and splitting diagonal runs eight bases at a time, which costs 1 to 3% of the alignment time.

Diagonal transition. Wavefronts are padded with unreached diagonals and advanced eight diagonals at a time with branch-free validity masks, in a separate function; a step costs about 1.3 ns, compared with 5 ns before. The two-ended search tests the overlap of the two wavefronts with vector comparisons and requires about half as many steps. Since a step costs about 1.3 ns and a band column about 6 ns, the diagonal transition is given a step budget proportional to the band it would replace; reducing this budget from 12 to 5 steps per column reduced the time for 10 kbp pairs at 5% divergence from 241 to 144 μ s. The choice between one and two wavefronts was fitted on 1,300 pairs of 1 to 30 kbp. For pairs of up to 2,048 columns, the band had been set wide across the whole matrix, and 42% of such pairs were assigned to a band although the diagonal transition would have been faster; such a band is now charged an additional 30,000 steps, which made the 1 kbp reads 25% faster on the Skylake-X. A short noisy pair abandons the diagonal search in favour of the full matrix (a 1 kbp pair at 15% divergence, 19.2 to 11.8 μ s), and the first bound of a short pair is 1.7 times its extrapolation, at most 128 above it.

Gather masks. A gather consumes its mask register, and compilers generate the required all-ones mask with `kxnor`, which depends on the previous value of that register; consecutive gathers were therefore serialised at about 21 cycles per group. Deriving the mask from the group’s own lanes removes this dependency. Gathered extension made the 1 kbp reads, the SARS-CoV-2 genomes and 100 kbp pairs at 1 to 2% divergence 7 to 19% faster (Table S1, commit 45697e0).

Measured thresholds. A full bit-parallel sweep of a 64-row word costs 35%, 45% and 77% of a diagonal-transition step on AVX-512, AVX2 and NEON, respectively, measured on 1 kbp pairs at 15% divergence (the AVX2 value on the Skylake-X with code compiled for Haswell). On AVX-512, seeds reduce the running time only from about 86 kbp: below that length, the Skylake-X aligned real reads of 16 to 64 kbp 14% faster without seeds, and uniform 30 kbp pairs at 5% divergence 30% faster, whereas at 100 kbp seeds reduced the time by 7 to 24%. With AVX2 and NEON, seeds are used from 16 kbp. For pairs whose two extrapolations agree, as with uniform errors, the inexact-seed threshold is 20% on every platform, and from 64 kbp an extrapolated error rate of at least one edit in ten bases selects inexact seeds directly.

Interleaving. `llvm-mca` estimates the interleaved AVX-512 loop at 24 cycles per column, against a throughput bound of 11, so the loop remains limited by the lane rotation. A third group still fits in the 32 AVX-512 registers, and `llvm-mca` predicted 15% fewer cycles per group, but the measured time changed by less than 1.5%.

S1.2 Seed heuristic

Batched lookups. Query rows are processed in batches of 16: the bucket bounds of all rows are loaded first, so that the loads overlap, and their entries are prefetched; each entry stores a seed together with its code, so that a single load provides both. A lookup tests each candidate window with a branch-free expression. Together with considering only seeds that can still chain to the end, these steps reduced the inexact setup on the ultra-long reads with genetic variation from 5.02 to 2.90 s in total. On 100 kbp pairs at 10% divergence, seed construction still accounts for 58% of the running time on the Skylake-X.

Admissibility check and correction. Comparing the heuristic with the true remaining cost at every cell of random pairs showed that an earlier version of our implementation overestimated the remaining cost by one on optimal paths in 1 to 2% of pairs. The cause was a filter that discarded matches whose gap to the end of the pair exceeded the potential of the intervening seeds: a match ending one diagonal beyond this limit was discarded, although a path through it costs only one more than the potential, and the neighbour that would otherwise bound such paths could be removed by local pruning. Such a match now chains to the end at the cost of its excess, entering layer $2 - 1 = 1$; no further match can follow it. After the correction, no cell exceeded the true cost in 2,200 random pairs, compared with 34,450 cells in 400 pairs before. The reported distances agreed with those of the other aligners in all benchmarks, both before and after the correction. The overestimate arose from our filter; we do not claim that it applies to A*PA or A*PA2.

Exact-seed filter. A 32-bit filter per seed, placed before a half-full open-addressing hash table, resolves most lookups with a single predictable test. On the M2, matching the 320 mid-length reads took 32 ms instead of 87 ms, and 30 kbp pairs at 5% divergence became 15% faster.

Local pruning. The diagonal-transition search used for local pruning updates its wavefronts in place, with an unreachable diagonal on either side replacing three bounds checks per diagonal.

S1.3 Gap-affine costs

Compact traceback. For the traceback of the bidirectional wavefront, `dinara-align` stores, for every cost and diagonal, the furthest-reaching column and one byte recording the predecessor of each layer: five bytes per diagonal, compared with twelve in the high-memory mode of WFA. When the stored wavefronts would exceed the memory limit (80 MB by default), the pair is split at a cell through which an optimal path passes. Under a general substitution matrix, substitution scores are obtained by byte shuffles (`pshufb`, `tbl`) for matrices of up to 16 entries and by gathers otherwise.

Gathers. In the gap-affine wavefront, AVX-512 gathers reduced the time for 1 kbp reads from 149 to 129 μ s. Deriving each gather’s mask from the group’s own lanes, with one reduction after the loop rather than one per

group, reduced it further to 111 μ s, and the time for 10 kbp reads from 2.34 to 1.84 ms. Extension along matches was moved into the wavefront step, with columns stored through scalar loads and stores rather than reinserted into a vector, and retained wavefronts are held in fixed blocks that are never moved; memory moves had accounted for 7.5% of profiling samples (Table S1).

Memory layout. The wavefront updates the bounds and furthest reaches of its slots at every cost. When these were separate arrays of a few words, the allocator placed them next to another thread’s data, so that the cache line moved between cores at every write; ten threads on the Skylake-X then required twice the cycles per pair of one thread, with 40 cache lines fetched from another core per pair. Each such array now occupies cache lines that no other allocation shares. A worker retains its profile, reversed sequences and search buffers between pairs, so that a warm worker performs no allocation; per-pair allocations had accounted for one third of the time of a short read, and caused contention in the allocator with many threads. Padding each row of a wavefront buffer by one cache line, to avoid 4 KB aliasing between loads and earlier stores, removed 1.5 billion blocked loads over ten 100 kbp pairs. Separating the working memory of threads by two cache lines rather than one reduced the cache lines fetched from other cores from 3.4 million to 0.3 million over 2 million reads on ten threads.

S1.4 Other modes

Extension. Extension alignment uses a single wavefront search from the anchor, with the match score folded into the costs, and records the best end point on the furthest anti-diagonal of each wavefront. The search stops once a proven bound shows that no higher cost can yield a better score, and the prefix up to the best end point is then aligned globally. With an end bonus, the search also records the best point on the last row of the query and continues only while such an alignment can still come within the bonus; this reduced the time for an extension with an end bonus from 1.02 ms to 186 μ s.

Ends-free alignment. Under gap costs with free ends, the forward search starts at cost zero on every diagonal on which a free start lies. The span is determined first: one search from the free side finds the end on the highest diagonal, and a narrow search backwards finds the start; the span is then aligned globally. On x86, this aligns a read at the start of a reference, an overlap or a suffix 1.4 to 3.5 times faster, and a read anywhere within the reference 0 to 14% slower, than the previous approach. Keeping the search state outside an optional wrapper while the search runs, so that its fields remain in registers, reduced the time for an infix alignment at (4, 6, 2) from 3.92 to 3.00 ms.

Infix and prefix alignment at unit costs. Infix alignment uses the bit-parallel sweep with the cut-off of Edlib, with a bound doubling from 64; words entering the band are initialised to +1, and the last word advances one column at a time. Prefix alignment sweeps only the band permitted by its bound: the top of the band moves down with the diagonal, no column beyond the read length plus the best end so far is computed, an attempt whose band becomes empty stops, and at one eighth, one quarter and one half of the columns an attempt that can no longer succeed is abandoned. This reduced the time for a prefix distance from 23.7 to 20.5 μ s on the Skylake-X.

A band derived from the score. A global alignment under a single match score and a single mismatch score stores only the diagonals permitted by its score. In the converted costs, every gapped letter contributes at least $2e + a$ to the total $C = a(n + m) - 2S$, and a path that deviates by t diagonals from both the main and the end diagonal contains at least $2t$ gapped letters, so only diagonals within $C/(2(2e + a))$ of these can contain an optimal path. For a 1 kbp pair at 5% divergence, 137 diagonals are stored instead of 1,000. Under any other substitution matrix, the score of the sweep bounds the band in the same way. The backward sweep from the end of a local alignment and each half of a linear-space alignment also use 16-bit lanes while the scores fit, which made 1 kbp local alignments 1.2 times and linear-space global alignments 1.4 times faster on the M2.

S1.5 Batches

Letter layout. The pairs of a group are interleaved so that one 64-bit load reads eight letters of a lane, and eight lanes are transposed in registers by two shuffles.

Alignment in 8-bit lanes. Batch alignments remain in 8-bit lanes, 64 pairs per AVX-512 register, and a pair whose cost exceeds the 8-bit range is passed to a single call. With 16-bit lanes, half as many lanes would sweep the wide band that such a pair requires, which on the Skylake-X took as long as the searches on 1 kbp reads at 10% divergence, and twice as long on NEON.

Cache-efficient traceback. Storing the traceback flags of a group’s whole band would exceed the caches on long pairs and touch new memory pages for every group; with ten threads, this doubled the time for 1 kbp pairs relative to the searches alone. The sweep therefore stores one row of every layer every s rows and recomputes a segment to obtain its flags during the traceback. With R rows, s is the square root of R times the bytes per cell of a stored row divided by those of a flag, which balances the stored rows against the flags of one segment and minimises memory use.

Lane groups. A group of 64 bytes occupies one register on AVX-512 and several on narrower vector units; with the 128-bit registers of the M2, 1 kbp pairs at 10% divergence required 73 μ s per pair under affine costs with four registers, compared with 132 μ s with one. Local scores use 32 lanes of 16 bits or 16 lanes of 32 bits, four

independent registers’ worth of work; on the M2, this scored 20,000 references against a read in 83 ms, compared with 160 ms with one register. Pairs whose first band does not satisfy the bound are grouped by the exact width of their second band, and a group stops once every lane has exceeded its cost limit.

S1.6 Code generation

Avoiding division by a variable. A 64-bit division by a variable takes tens of cycles on x86. The potential of the heuristic, required at every row that the band prunes, involves a division by the seed length, so each seed length has its own code path with a constant divisor, which compiles to a multiplication; batches likewise avoid a division per pair.

Kernels compiled separately. When inlined next to each other, or selected per band round, the bit-parallel kernels for plain bases and for additional symbols slowed the plain kernel by 1 to 4%; when inlined into the seed construction, the local-pruning search slowed the construction loop by a few percent. Separate seeded and unseeded versions of the band keep the code of the heuristic out of unseeded pairs, which it had slowed by 3 to 7%.

Operand order. Under a substitution matrix, the recurrence first takes the maximum of the two gap layers, so that the substitution score, the operand that arrives last, waits on only one maximum; a score obtained by comparison is available early and is used first. This was measured on the M2.

Branches. On a 1 Mbp pair at 15% divergence, branch mispredictions account for about 7% of the cycles, mostly at short loop exits, compared with 2 to 3% in the bit-parallel sweep. A fully branch-free seed scan would spend about 25 million cycles per pair to save about 34 million, a gain of 1 to 2%, and was not adopted.

S2 Measured effect of each change

Table S1: Changes behind the results and their measured effects, as recorded when each was made. Machines: the i9-7900X (Skylake-X, AVX-512) and the Apple M2 (NEON); “not recorded” where the commit does not say.

Commit	Change	Measured effect	Machine
a6101e5	Inexact seeds ($r = 2$), matched by halves	100 kbp 15.6 $\rightarrow$ 9.5 ms; 300 kbp 105 $\rightarrow$ 47 ms; 1 Mbp 1.36 $\rightarrow$ 0.41 s	M2
7435edd	Leaner seed setup: branch-free test, reachable seeds only, fronts in place	inexact setup on genvar 5.02 $\rightarrow$ 2.90 s; genvar 11.77 $\rightarrow$ 10.21 s; ont-500k 8.04 $\rightarrow$ 7.11 s	Skylake-X
28b9e67	Inexact-seed cut-off by vector width (20% on AVX-512)	ont-500k (50 reads) 8.73 $\rightarrow$ 8.03 s; genvar (48) 12.03 $\rightarrow$ 11.73 s; 100 kbp at 6% 114 $\rightarrow$ 64 ms	Skylake-X
708f151	Seeded retry a quarter of the climb past the estimate	genvar 9.86 $\rightarrow$ 9.66 s; ont-500k 6.81 $\rightarrow$ 6.30 s; 100 kbp at 15% 4% faster	Skylake-X
ff26cea	Seed gate at 86 kbp on AVX-512; exact-seed filter	30 kbp at 5% 1.48 ms $\rightarrow$ 898 μ s; SARS-CoV-2 373 $\rightarrow$ 283 μ s; ont-50k 897 $\rightarrow$ 800 μ s	Skylake-X
ff26cea	Exact-seed filter alone	matching on 320 mid-length reads 87 $\rightarrow$ 32 ms; 30 kbp at 5% 15% faster	M2
45697e0	AVX-512 gathers in the diagonal transition	ont-1k 26.8 $\rightarrow$ 25.0 μ s; SARS-CoV-2 335 $\rightarrow$ 306 μ s; 100 kbp at 1% 1.70 $\rightarrow$ 1.45 ms, at 2% 5.60 $\rightarrow$ 4.54 ms	Skylake-X
2c7c46e	Short pairs stay in the diagonal transition	ont-1k 42.3 $\rightarrow$ 31.8 ms over 1,221 reads	Skylake-X
e0ea104	Alignment step budget 12 $\rightarrow$ 5 a column	10 kbp at 5% 241 $\rightarrow$ 144 μ s	not recorded
a593ade	Short noisy pairs give up the diagonal search	1 kbp at 15% 19.2 $\rightarrow$ 11.8 μ s	Skylake-X
beb7a8c	Tiles traced by a forward search	ont-50k 3% and ont-500k 8% faster	M2
5a0e514	Tile search eight diagonals at a time, gathered	100 kbp at 15% tile searches 3.55 $\rightarrow$ 1.4 ms a pair	Skylake-X
7f1a813	Windowed recompute of a failed tile	600 kbp at 13% 1.38 s $\rightarrow$ 617 ms	not recorded
3becb4b	Recompute buffers reused, uninitialised	ont-1k 23.4 $\rightarrow$ 22.6 μ s; ont-10k 151 $\rightarrow$ 142 μ s	Skylake-X
e2125b0	Tile edges sized once, written through pointers	ont-500k 2 to 3%, ont-10k about 2% faster	both
38837f2	Fronts sixteen offsets a store; unchecked reads	ont-1k 21.4 $\rightarrow$ 20.7 μ s; SARS-CoV-2 213 $\rightarrow$ 199 μ s	Skylake-X
34cb623	Pointers in hot lookups	genvar 10.16 $\rightarrow$ 9.90 s; ont-500k 7.04 $\rightarrow$ 6.82 s	Skylake-X
f29a945	Separate seeded and unseeded band copies	unseeded pairs back within 2% (had slowed 3 to 7%)	Skylake-X
aca56c7	AVX-512 gathers in the gap-affine step	ont-1k 149 $\rightarrow$ 129 μ s; ont-10k 3.52 $\rightarrow$ 2.94 ms; 10 kbp at 15% 36.4 $\rightarrow$ 24.5 ms	Skylake-X
53e15d4	Gather mask from the group’s own lanes; one reduction after the loop	ont-1k 129 $\rightarrow$ 111 μ s; ont-10k 2.34 $\rightarrow$ 1.84 ms; 10 kbp at 15% 17.9 $\rightarrow$ 14.0 ms	Skylake-X
ced0b8d	Ring rows padded against 4 KB aliasing	ont-10k 1.86 $\rightarrow$ 1.68 ms; 100 kbp at 5% 349 $\rightarrow$ 305 ms	Skylake-X
17fd781	Slides straight from the step	100 kbp at 5% 315 $\rightarrow$ 287 ms; ont-10k 1.80 $\rightarrow$ 1.67 ms	M2

Table S1: (continued)

Commit	Change	Measured effect	Machine
01c1e9b	Slide columns through scalar loads and stores	100 kbp at 5% 287 → 271 ms; ont-1k 91 → 86 µs	M2
72c3696	Kept fronts in fixed blocks, never moved	ont-10k 1.48 → 1.37 ms (memmove had been 7.5% of samples)	M2
b7e5232	Gap-affine traceback through the wavefront’s own fronts	SARS-CoV-2 1.34 ms → 638 µs; ont-10k 14.6 → 4.05 ms	M2
e6a435f	End bonus weighed in the extension’s own search	with an end bonus 1.02 ms → 186 µs (KSW2 1.23 ms)	Skylake-X
8ba2a6a	Free ends: span first, start at cost zero on free diagonals	start, overlap, suffix 1.4 to 3.5× faster; anywhere 0 to 14% slower	Skylake-X
f0bb161	Unit-cost infix: Ukkonen cutoff, restarted words	10 kbp in 100 kbp 21 → 13 ms; 100 kbp in 1 Mbp 2 → 0.75 s	M2
2f8b935	Prefix search abandons a try that cannot succeed	distance 23.7 → 20.5 µs; alignment 45.8 → 42.9 µs	Skylake-X
2b038c8	Global band bounded by the score (one match, one mismatch score)	1 kbp at 5%: 137 of 1,000 diagonals; 0.36 → 0.21 ms	not recorded
756f3e1	Table sweeps vectorised, any table	1 kbp global 13.3 ms → 550 µs; local 133 → 2.89 ms	Skylake-X
433a8b5	Local under gap costs: lanes along the shorter sequence, extension back	47 vs 55 µs short pairs; 1.93 vs 3.22 ms 1 kbp in 10 kbp (SSW)	Skylake-X
cf87b2b	Local sweep in 16-bit lanes	3.4 ms, from 4.6	Skylake-X
a8355aa	Batch letters transposed in registers	affine 0.59 → 0.43 µs a pair; unit 0.55 → 0.40 µs	not recorded
20425a7	Batch first pass in saturating bytes	affine 0.43 → 0.40 µs a pair	Skylake-X
64f0cc3	Second-pass bands grouped by exact width	1 kbp at 10% scores 18 → 15 µs; 46 → 38 µs at (4, 6, 2)	not recorded
0f7b5af	A lane group stops once every lane has passed its cap	1 kbp at 10% aligns 33 → 22 µs, scores 18 → 12 µs	Skylake-X
792ba9f	Pairs with N take seeds; a seed with N goes uncounted	500 kbp reads 2.5 to 3× faster (M2), 1.4 to 1.7× (Skylake-X)	both

S3 Additional results

Table S2 gives the results for the other alignment modes, each compared on one pinned thread of the Skylake-X with the aligners that support the same mode. Table S3 gives the complete gap-affine comparison at costs (4, 6, 2). At 5% divergence, dinara-align aligned uniform pairs of 3 kbp to 1 Mbp in 42 µs, 284 µs, 941 µs, 4.85 ms, 19.4 ms and 63.3 ms, compared with 174 µs, 788 µs, 2.45 ms, 8.22 ms, 27.6 ms and 125 ms for A*PA2-full.

Table S4 gives the mean and median time per alignment on the real datasets of the A*PA2 benchmark, including the two approximate aligners of that benchmark, WFA-adaptive and Block Aligner, run with its parameters. For each approximate aligner, the percentage of optimal alignments is given beside its time, computed against an exact alignment at the same costs.

S4 Robustness

Three changes address inputs on which the running time had been disproportionate:

- **Low-complexity sequence.** Wavefront layers that spread into a staircase pattern are handled explicitly, and the pruning search is bounded; a poly-A sequence of 32,000 bases aligned against one of 16,000 bases took 0.15 s instead of 17 s.
- **Repetitive seeds.** Seeds are abandoned when they exceed 64 matches per seed on average; setting up a 200 kbp tandem repeat took 0.045 s and 34 MB instead of 73 s and 760 MB.
- **Unreachable wavefronts.** A wavefront that reaches no cell is not tested further; one such case took 13 ms instead of 1.5 s.

S5 Approaches evaluated and not adopted

- **Multi-threading within a pair.** Striped bands over eight threads aligned 1 Mbp pairs at 15% divergence in 575 ms, compared with 1,122 ms on one thread. The implementation was removed because the library runs on the calling thread and distributing pairs over threads yields a larger gain.
- **A third bit-parallel group on AVX-512.** The measured time changed by less than 1.5%.
- **One sixteen-lane group instead of two eight-lane groups.** Equal on long reads, and up to 2% slower on short divergent pairs.

Table S2: Other modes: mean time per pair on one thread. Every tool’s scores agree on every pair.

Workload	dinara-align	Next fastest
Local, 600 short noisy pairs (20–700 bp)	32 μs	SSW 52 μ s
Local, 1 kbp read at 10% in a 10 kbp window	1.40 ms	SSW 3.19 ms
Local, 10 kbp at 5% against 12 kbp	50.5 ms	SSW 85.0 ms
Overlap of 2 kbp reads by 0.5–1.5 kbp	2.15 ms	parasail 15.3 ms
Infix with a reward, 1 kbp read in a 3 kbp window	1.40 ms	parasail 14.8 ms
Infix at unit costs, 1 kbp read in a 3 kbp window	135 μs	Edlib 270 μ s
Infix at unit costs, 10 kbp read in a 30 kbp window	3.14 ms	Edlib 13.4 ms
Prefix at unit costs, 1 kbp read against 2 kbp	43 μs	WFA2-lib 47 μ s
Infix at (4, 6, 2), 1 kbp read in a 3 kbp window	3.00 ms	WFA2-lib 4.10 ms
Extension, match 2 at (4, 6, 2)	726 μs	KSW2 2.07 ms
Extension with an end bonus of 50	158 μs	KSW2 1.23 ms
Two-piece gaps (4, 6, 2, 24, 1), 5 kbp	8.44 ms	WFA2-lib 15.9 ms
Table, global, 1 kbp at 10%	542 μs	parasail 2.94 ms
Table, local, 1 kbp read in a 10 kbp window	2.45 ms	SSW 3.27 ms

- **A fully branch-free seed scan.** It would save 1 to 2% at a cost of about 25 million cycles per pair.
- **A Bloom filter for inexact keys.** It halved the number of candidates, but reduced the alignment time by only 1.5 to 2%.
- **Shorter look-ahead, or no local pruning.** Without pruning, the ultra-long reads with and without genetic variation were 21% and 72% slower, respectively.
- **A* search within the diagonal transition, and an upper bound from beam search.** Each required several tuned parameters for small gains.
- **Traceback during the sweep.** The stored tile boundaries occupy 0.6 to 13 MB, compared with 10 to 17 MB for the sequences and bit planes, so little memory could be saved.

Table S3: Gap-affine alignment at costs (4, 6, 2), all datasets: mean time per alignment with a 5 s budget per sample. Superscripts give the pairs finished where not all were; no tool finished any pair of the ultra-long reads or of 1 Mbp pairs, nor of 300 kbp at 15%.

Dataset	dinara-align	WFA	BiWFA	KSW2
ont-1k	112 μs	186 μ s	304 μ s	1.1 ms
ont-10k	1.68 ms	4.8 ms	5.84 ms	30.8 ms ^{163/277}
ont-50k	21.2 ms	68.4 ms ^{82/104}	57.3 ms ^{90/104}	367 ms ^{16/104}
SARS-CoV-2	755 μs	2.06 ms	1.79 ms	88.4 ms
3 kbp, 5%	283 μs	579 μ s	913 μ s	6.12 ms
10 kbp, 5%	2.17 ms	6.36 ms	8.02 ms	97.1 ms ^{52/99}
30 kbp, 5%	18.1 ms	56 ms	62.7 ms	1.35 s ^{4/33}
100 kbp, 5%	304 ms	754 ms ^{7/10}	643 ms ^{8/10}	—
300 kbp, 5%	3.07 s^{1/4}	—	—	—
3 kbp, 15%	1.26 ms	3.6 ms	5.08 ms	18.7 ms ^{268/333}
10 kbp, 15%	12.2 ms	38.5 ms	44.8 ms	203 ms ^{25/99}
30 kbp, 15%	151 ms	385 ms ^{13/33}	372 ms ^{14/33}	2.48 s ^{2/33}
100 kbp, 15%	2.02 s^{2/9}	—	4.19 s ^{1/9}	—

Table S4: Real datasets of the A*PA2 benchmark: mean time per alignment, with the median in parentheses. Superscripts give the number of pairs completed within 20 s where not all were; a dash marks a tool that completed none. For the approximate aligners (*), the percentage of optimal alignments is given.

Dataset	dinara-align	A*PA2-full	A*PA2-simple	A*PA	Edlib	BiWFA	WFA	WFA-adaptive*	Block Aligner*
ont-1k	23 μs (23 μs)	81 μ s (89 μ s)	48 μ s (54 μ s)	612 μ s (556 μ s)	126 μ s (120 μ s)	44 μ s (46 μ s)	31 μ s (32 μ s)	41 μ s (42 μ s), 93%	41 μ s (47 μ s), 85%
ont-10k	163 μs (111 μs)	391 μ s (264 μ s)	247 μ s (177 μ s)	15.3 ms (3.29 ms)	1.1 ms (579 μ s)	753 μ s (284 μ s)	595 μ s (223 μ s)	385 μ s (172 μ s), 61%	198 μ s (141 μ s), 61%
ont-50k	587 μs (242 μs)	1.27 ms (513 μ s)	944 μ s (307 μ s)	190 ms (10.5 ms) ^{81/104}	6.1 ms (2.02 ms)	7.43 ms (875 μ s)	6.37 ms (722 μ s)	2.32 ms (459 μ s), 52%	657 μ s (319 μ s), 61%
ont-500k	104 ms (83.8 ms)	176 ms (113 ms)	537 ms (371 ms)	—	4.99 s (3.04 s) ^{4/15}	19.8 s (19.8 s) ^{1/15}	—	989 ms (629 ms), 53%	728 ms (714 ms)
ont-500k-genvar	144 ms (114 ms)	210 ms (177 ms)	624 ms (506 ms)	—	5.23 s (5.2 s) ^{4/15}	6.36 s (7.57 s) ^{3/15}	—	533 ms (273 ms), 7%	836 ms (647 ms), 0% of 1
SARS-CoV-2	279 μs (188 μs)	2.00 ms (2.07 ms)	728 μ s (662 μ s)	6.49 ms (2.07 ms)	8.15 ms (7.72 ms)	895 μ s (446 μ s)	983 μ s (359 μ s)	611 μ s (353 μ s), 97%	2.51 ms (2.3 ms), 30%